

Session 9: Modern Generative AI Transformers & Large Language Models

HuggingFace Ecosystem · Fine-Tuning LLMs · Retrieval-Augmented Generation

Javier Serrano

Applied Machine Learning

Master in Data Science and Analytics



Strathmore University

@iLabAfrica Centre

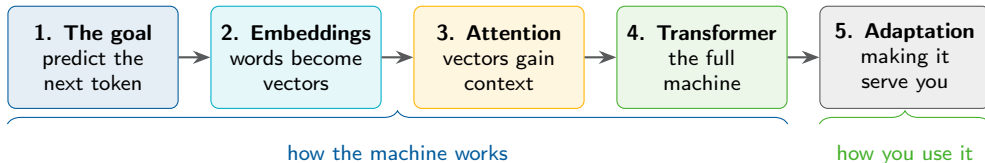
Learning objectives

By the end of this session you will be able to:

1. State what a language model computes, and why one objective produces many abilities.
2. Explain what an embedding is, how it is trained, and why its axes carry meaning.
3. Derive attention as a soft lookup, and give the shape of every matrix involved.
4. Describe a Transformer block: attention, MLP, residuals, and the final softmax.
5. Choose between prompting, retrieval and fine-tuning for a given task.

Takeaway: Five ideas, each one built on the previous one. If any step is unclear, stop me there.

The path through this session



Takeaway: Each box answers a question the previous box leaves open.

Roadmap

The goal: predict the next word

Embeddings: words become vectors

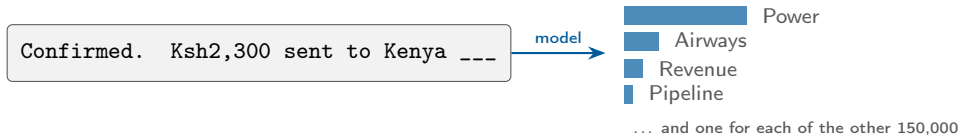
Attention: adding context

The Transformer: the full machine

Adapting a pretrained model

A language model has exactly one job

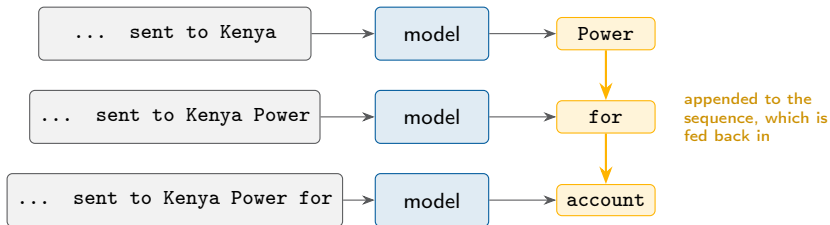
Given a sequence of words, output a **probability for every possible next word**.



$$p_{\theta}(x_t \mid x_1, \dots, x_{t-1}) \quad \text{for every token in the vocabulary}$$

Takeaway: Not classification, not translation, not chat. One distribution over the vocabulary, at every position.

Writing text is that job in a loop



Sample one token, glue it onto the end, feed the whole thing back. Repeat.

Takeaway: Everything an LLM appears to do (answer, summarise, translate, reason) is this loop. The skill is in the probabilities, not in the loop.

The units are tokens, not words

The vocabulary is a fixed list of $V \approx 150,000$ **subword** pieces, learned by merging frequent character pairs (BPE).

"Kenya Power" $\longrightarrow$ Kenya Power 2 tokens, 2 words

"Nimetuma pesa" $\longrightarrow$ Ni met uma pes a 5 tokens, 2 words

Fertility = tokens per word. English ≈ 1.3 ; Swahili and Sheng often 3 to 4 on English-centric vocabularies.

Takeaway: High fertility costs you three times: a larger bill per message, less content inside the context window, and weaker learning signal per word.

Why one boring objective produces capability

To predict the next token well over trillions of tokens, the model is forced to internalise whatever makes text predictable:

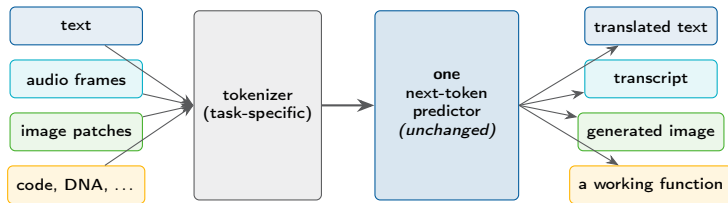
- "The customer failed to repay the" → needs **syntax**.
- "Ksh2,300 sent to Kenya" → needs **world knowledge**.
- "12 loans at Ksh 4,500 each is Ksh" → needs **arithmetic**.
- "def compute_interest(principal, rate):" → needs **code semantics**.

Minimising $-\sum_t \log p_\theta(x_t \mid x_{<t})$ is the same as compressing the corpus into as few bits as possible.

Takeaway: Prediction is compression. Abilities are not trained for individually; they are what good compression looks like from outside.

The same objective, many applications

Anything you can serialise into a sequence of tokens becomes next-token prediction. Only the *tokenizer* changes.



Translation: predict the French continuation of an English prompt. **Speech recognition** (Whisper): audio becomes tokens, predict the transcript. **Text to image:** predict image patch tokens, then decode them to pixels. **Code:** predict the next line.

Takeaway: The architecture was never specialised for language. It is a general sequence predictor, and that is why one idea absorbed so many separate research fields.

The problem this leaves us with

To compute $p_{\theta}(x_t \mid x_{<t})$ the model needs a numeric representation of each word. Two requirements pull in opposite directions:

Requirement 1

A word must have a **stable** meaning the model can learn once and reuse everywhere.

Requirement 2

The same word must mean **different things** in different sentences. “Sent” in a payment SMS is not “sent” in a parcel email.

The architecture answers these with two separate mechanisms: **embeddings** give stable meaning, **attention** makes it context-dependent.

Takeaway: The rest of the session is those two mechanisms, in that order.

Roadmap

The goal: predict the next word

Embeddings: words become vectors

Attention: adding context

The Transformer: the full machine

Adapting a pretrained model

The embedding table is a lookup table of parameters

One row per token in the vocabulary:

$$\mathbf{E} \in \mathbb{R}^{V \times d}, \quad V \approx 150,000, \quad d = 4096$$

- That is about 600 million numbers, all of them **trainable**.
- “Looking up” token i means taking row i .
No computation at all.
- A sentence of T tokens becomes $\mathbf{X} \in \mathbb{R}^{T \times d}$, one row per position.

$d = 4096$ columns

Ksh					
sent	0.31	-1.04	0.88	...	0.12
loan					

V rows

Takeaway: The embedding layer is not a neural network. It is $V \times d$ numbers that gradient descent slowly edits.

Meaning lives in directions, not in single numbers

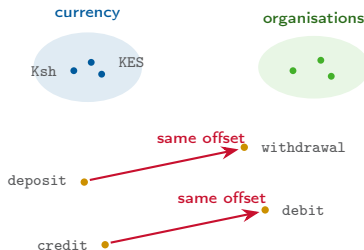
In 4096 dimensions, **directions** carry meaning:

$$e_{\text{king}} - e_{\text{man}} + e_{\text{woman}} \approx e_{\text{queen}}$$

Two structures appear in our corpus:

- **Proximity** is synonymy: Ksh and KES land almost on top of each other.
- **A shared offset** is a relation: “reverse the direction of money” is the same arrow wherever it applies.

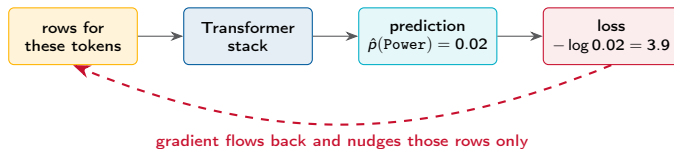
No axis was labelled by anyone; the structure emerges from training.



Takeaway: The space is hyperdimensional so that thousands of independent semantic distinctions can coexist without colliding.

How the table gets trained

Nothing trains embeddings directly. The gradient of the *next-token loss* reaches back into the table.

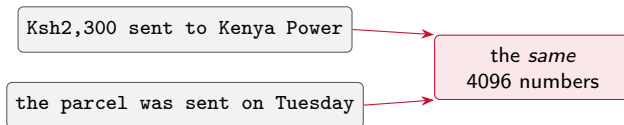


Each nudge is tiny, but “sent” appears in hundreds of millions of fragments. Two tokens that occur in *similar contexts* receive *similar gradients*, so their rows drift together.

Takeaway: Nobody tells the model that Ksh and KES are related. They converge because the contexts they live in are the same.

The limitation that motivates everything next

An embedding row is fixed once training ends. Row 8,412 is the same vector for “sent” in every sentence ever written:



The embedding gives “sent in general”, an average over every use. What we actually need is “sent, here, meaning Ksh2,300 going to Kenya Power”.

Takeaway: Embeddings supply context-free meaning. Attention exists to finish the job.

Ahead: the next section builds exactly that mechanism.

Roadmap

The goal: predict the next word

Embeddings: words become vectors

Attention: adding context

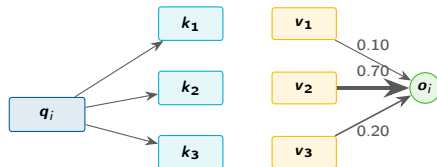
The Transformer: the full machine

Adapting a pretrained model

From a hard lookup to a soft one

A dictionary lookup is all-or-nothing: the key matches or it does not.

- Nudge the query slightly and the output either does not move at all, or jumps completely.
- No notion of “getting warmer”, so **no gradient** and nothing to train.



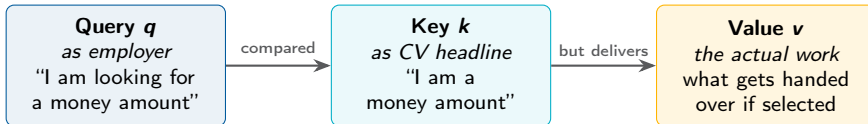
weights are smooth, so gradients exist

Attention scores every key, turns the scores into percentages, and returns a **blend** of all the values.

Takeaway: Softmax replaces an if-statement with a dial, and a dial is something gradient descent can turn.

Queries, keys and values: three roles, one word

Think of each word at a hiring fair, playing two roles at once:



Matching happens between **query and key**. What is delivered is the **value**. Keys are for *finding*; values are for *content*.

Takeaway: Separating key from value lets a word advertise itself one way and contribute something else.

The three projections, with shapes

Each of the three roles is a learned linear map of the same input $\mathbf{X}$:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}^V$$

Object	Shape	Meaning of a row
$\mathbf{X}$	$T \times d$ (T tokens, $d = 4096$)	one token's current vector
$\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$	$d \times d_k$ ($d_k = 128$)	<i>learned</i> , shared by all positions
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	$T \times d_k$	one token's query / key / value

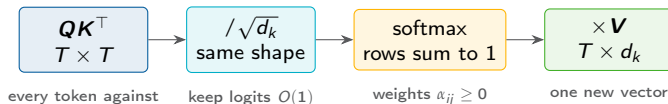
Only the three $\mathbf{W}$ matrices are parameters. $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ are recomputed for every sentence.

Takeaway: The model does not compare words. It compares *learned projections* of words, which is how “relevant” becomes something it can discover.

Scaled dot-product attention

Score every query against every key, normalise, and blend the values:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$



Output row i is $\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j$, with $\sum_j \alpha_{ij} = 1$.

Takeaway: Because the weights are positive and sum to one, the output is always a blend of the values, never an extrapolation beyond them.

Worked example: what does “sent” need to know?

Three tokens, two toy dimensions (*amount, organisation*), $d_k = 2$:

Token	query q	key k	value v
Ksh2,300	(0, 0)	(1, 0)	(10, 0)
sent	(1, 1)	(0, 0)	(0, 0)
Kenya Power	(0, 0)	(0, 1)	(0, 10)

The verb is the only token shopping: it wants an amount *and* an organisation. Scoring its query, then dividing by $\sqrt{2}$ and applying softmax:

$$(1, 1) \cdot (1, 0) = 1, \quad (1, 1) \cdot (0, 0) = 0, \quad (1, 1) \cdot (0, 1) = 1 \Rightarrow \alpha = (0.40, 0.20, 0.40)$$

$$\mathbf{o}_{\text{sent}} = 0.40 (10, 0) + 0.20 (0, 0) + 0.40 (0, 10) = (4.0, 4.0)$$

Takeaway: The verb arrived carrying nothing and leaves carrying both the amount and the counterparty. That is what “adding context” means, concretely.

Why divide by $\sqrt{d_k}$

If $\mathbf{q}$ and $\mathbf{k}$ have independent unit-variance components, then

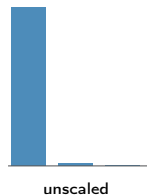
$$\text{Var}(\mathbf{q}^\top \mathbf{k}) = d_k$$

With $d_k = 64$ the scores have standard deviation 8, so softmax saturates:

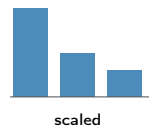
- one weight takes 99.6% of the mass;
- the gradient scales like $p(1-p) = 0.004$;
- training stalls.

Dividing by $\sqrt{d_k} = 8$ restores unit scale.

saturated: no gradient



soft: learns

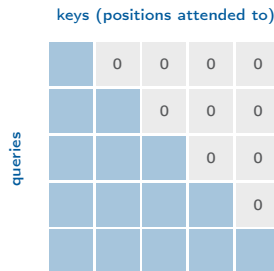


Takeaway: The $\sqrt{d_k}$ is not cosmetic. Without it the layer is not trainable.

Causal masking: one line separates reading from writing

Before the softmax, set $s_{ij} = -\infty$ for $j > i$,
so $\alpha_{ij} = 0$: token i cannot see the future.

- Without the mask the model reads both directions (an **encoder**: BERT, embedding models).
- With the mask it predicts left to right (a **decoder**: GPT, Llama, Qwen).
- The mask is free: all T positions still train in parallel, in one matrix product.



Takeaway: Masking is what makes $p_{\theta}(x_t \mid x_{<t})$ literally true, rather than an approximation.

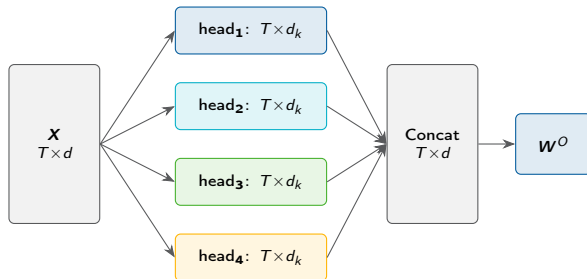
Multi-head attention: several relations at once

One attention map encodes *one* notion of relevance. Run $h = 32$ of them in parallel subspaces with $d_k = d/h$:

$$\text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O$$

- Each head: $T \times d_k$.
- Concatenated: $T \times d$.
- $\mathbf{W}^O \in \mathbb{R}^{d \times d}$ mixes them back.

Same total compute: width d is *sliced*, not added to.

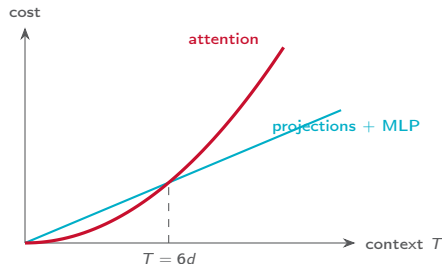


Takeaway: Heads specialise: some track syntax, some bind an amount to its currency, some link a payment to its counterparty.

The price: cost grows with the square of the context

The score matrix has T^2 entries, so attention costs $O(T^2d)$ time and $O(T^2)$ memory.

- Per layer, projections and MLP cost $12Td^2$; attention costs $2T^2d$.
- Attention passes half the layer only when $T > 6d \approx 24,500$ tokens.
- So a short SMS ($T = 64$) is free; a long retrieval prompt is where the bill lives.



Takeaway: Long context is affordable up to a point, and then suddenly is not. Retrieve five clauses rather than pasting sixty pages.

Roadmap

The goal: predict the next word

Embeddings: words become vectors

Attention: adding context

The Transformer: the full machine

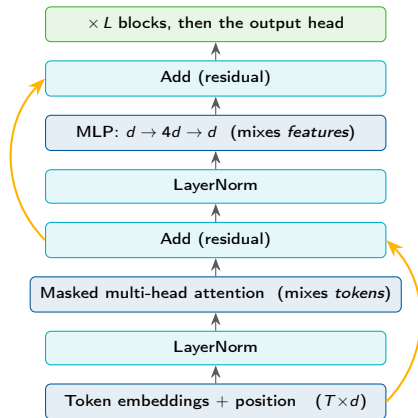
Adapting a pretrained model

One block, two sublayers

Every layer does exactly two things, in this order:

1. **Attention** mixes information *across tokens*. This is the only place where positions talk to each other.
2. **MLP** mixes information *across features*, each token independently. No token ever sees another here.

Each is wrapped in a residual connection and a LayerNorm. Stack $L = 32$ identical blocks.



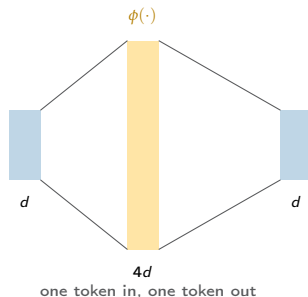
Takeaway: Attention moves information sideways; the MLP processes it in place. Alternating the two is the whole architecture.

What the MLP layers are for

Applied to each token separately, with ϕ a GELU or SwiGLU nonlinearity:

$$\text{MLP}(\mathbf{x}) = \mathbf{W}_2 \phi(\mathbf{W}_1 \mathbf{x}), \quad \mathbf{W}_1 \in \mathbb{R}^{4d \times d}, \quad \mathbf{W}_2 \in \mathbb{R}^{d \times 4d}$$

- **Expand to $4d$:** a much wider space, where many features become individually readable.
- **Nonlinearity:** keep some, suppress others. Without it the whole stack collapses into one matrix.
- **Project back to d :** write the surviving conclusions into the token's vector.



Takeaway: Two thirds of a Transformer's parameters sit in the MLPs. Attention decides *what to look at*; the MLPs hold most of *what it knows*.

Residuals and LayerNorm, in one slide

Residual: $\mathbf{x} \leftarrow \mathbf{x} + \text{Sublayer}(\mathbf{x})$

Each sublayer writes a *correction* onto a running vector rather than replacing it. The identity path gives the gradient a route that skips every nonlinearity, which is what makes 32 to 100 layers trainable.

LayerNorm

Rescales each token's vector to zero mean and unit variance before the sublayer reads it, so that activations cannot drift in scale as depth grows.

A useful mental image: the token's vector is a **notepad** passed up through the stack. Attention writes “here is context from other tokens”; the MLP writes “here is what that implies”. Nothing is ever erased.

Takeaway: Both mechanisms are borrowed unchanged from ResNets (Session 7). Nothing new was invented for Transformers here.

The last layer: back to a probability over the vocabulary

After L blocks, take the vector at the *final position* and project it to vocabulary size:

$$\mathbf{z} = \mathbf{h}_T \mathbf{W}_U, \quad \mathbf{W}_U \in \mathbb{R}^{d \times V}, \quad \mathbf{p} = \text{softmax}(\mathbf{z}/\tau) \in \mathbb{R}^V$$



Temperature τ reshapes the same logits: $\tau \rightarrow 0$ is deterministic (use it for compliance answers), $\tau \approx 0.8$ is varied (use it for drafting letters).

Takeaway: $\mathbf{W}_U$ is often the embedding table reused, $\mathbf{E}^\top$. The same vector that represents a token going in is what you compare against coming out.

Where the parameters actually are

A 7B model with $d = 4096$, $L = 32$, $V = 150,000$:

Component	Per layer	Total
Attention (W^Q, W^K, W^V, W^O)	$4d^2 \approx 67\text{M}$	$\approx 2.1\text{B}$ (30%)
MLP (W_1, W_2)	$8d^2 \approx 134\text{M}$	$\approx 4.3\text{B}$ (61%)
Embedding table E	(once)	$Vd \approx 0.6\text{B}$ (9%)

This matters for the next section: when you adapt a model cheaply, targeting the attention projections is small and fast, while also targeting the MLPs is more expressive and roughly doubles the cost.

Takeaway: Attention gets the attention. The MLPs get the parameters.

Roadmap

The goal: predict the next word

Embeddings: words become vectors

Attention: adding context

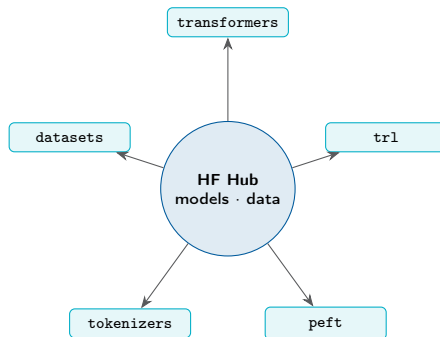
The Transformer: the full machine

Adapting a pretrained model

The HuggingFace ecosystem in one slide

One interface over roughly two million public models, the way scikit-learn is one interface over classical models.

- transformers: architectures, weights, Trainer.
- datasets: memory-mapped, streams corpora larger than RAM.
- tokenizers: fast BPE.
- peft: LoRA and QLoRA adapters.
- trl: SFT and preference tuning.



Takeaway: Pin the exact revision hash of every model you load. “Latest” is not a version, and a silently updated checkpoint is an unreproducible experiment.

Three ways to make a frozen model serve you

1. Prompting

weights frozen,
behaviour steered by
the context window

minutes

2. Retrieval

knowledge injected at
inference from your
own documents

an index, plus latency

3. Fine-tuning

a small set of new
weights trained on
your labelled data

GPU hours

Try them in that order, and escalate only when a measurement says to. They compose: production systems usually prompt a fine-tuned model over retrieved context.

Takeaway: Fine-tuning teaches a *skill* or a *format*. Retrieval supplies *facts*. Confusing the two is the most common design error.

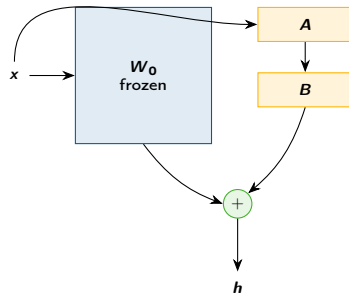
Fine-tuning cheaply: LoRA and QLoRA

Full fine-tuning of a 7B model needs about 16 bytes per parameter with Adam, so ≈ 112 GB. Instead, freeze $\mathbf{W}_0$ and learn a thin correction:

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B} \mathbf{A}$$

$$\mathbf{A} \in \mathbb{R}^{r \times k}, \quad \mathbf{B} \in \mathbb{R}^{d \times r}, \quad r = 16$$

For $d = k = 4096$ that is **0.78%** of the matrix. Quantising the frozen base to 4 bits (QLoRA) brings the whole job to 6 to 10 GB.

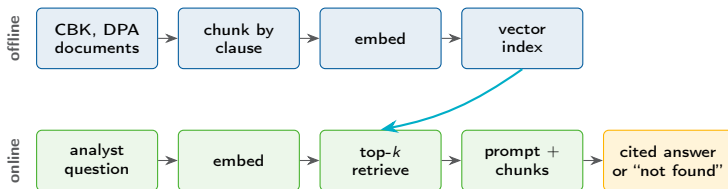


trainable: amber only

Takeaway: An 80 MB adapter, one consumer GPU, and the training data never leaves the country. That last point is why this matters here.

Retrieval-augmented generation

A model hallucinates, goes stale, and cannot cite its sources, because its knowledge is baked into the weights. RAG separates the two:



Updating knowledge becomes a document upload, not a training run, and every claim points at a clause.

Takeaway: Measure retrieval and generation *separately*, and measure refusals: a system that refuses everything scores perfectly on faithfulness.

Summary

1. A language model outputs one probability distribution over the vocabulary, at every position. Generation is that, in a loop.
2. **Embeddings** ($\mathbf{E} \in \mathbb{R}^{V \times d}$) give each token a stable vector whose *directions* carry meaning, learned by gradient descent on the same next-token loss.
3. **Attention** makes those vectors context-dependent: a soft lookup, $\text{softmax}(\mathbf{QK}^\top / \sqrt{d_k}) \mathbf{V}$, differentiable precisely because it blends instead of selecting.
4. A **Transformer block** alternates attention (across tokens) with an MLP (across features), joined by residuals, and ends in a softmax over V .
5. **Adaptation** goes prompting, then retrieval, then fine-tuning, and you escalate only on evidence.

Takeaway: Prompt first, ground second, fine-tune third, and measure at every step.

References

Main reading

Course book, Chapter 9: Modern Generative AI, Transformers and Large Language Models. The chapter follows this session section by section, with the full derivations, the code and the exercises.

Recommended video: , 3Blue1Brown *LLM explained* The clearest visual explanation of the mechanism available. Watch "LLM explained briefly" and chapters 5,6 and 7.

Papers: Vaswani et al. (2017) *Attention Is All You Need*; Hu et al. (2021) *LoRA*; Dettmers et al. (2023) *QLoRA*; Lewis et al. (2020) *RAG*; Hoffmann et al. (2022) *Chinchilla*.

Textbooks: Bishop & Bishop Ch. 12; Géron Ch. 16.